
Feature Learning in Supervised AI Models

Anonymous Author(s)

Affiliation

Address

email

Abstract

In this research paper we will learn and investigate how various supervised learning models learn features. We will do this through four tasks. In task 1 we will see how strong inductive bias is in models and how their architecture can effect how they learn features (i.e. whether they learn shape or texture or some other attribute). In task 2 we will observe how this effects Domain Generalization. The idea being that we want the model to learn identifying features about the objects (like their shape) instead of Domain Specific features (like if the dataset had grass behind every cat and the model associated the grass with the cat). If the model can do this, it should perform well even on Domain Shifts. We extend this idea in task 3 with Domain Generalization and completely unknown target domains. Finally, in task 4 we tackle the open set problem, to see if we can identify casual features even on unknown classes, and separate them as they come.

1 Introduction

We will do various experiments in this research paper to study how different AI models learn features, and the consequence of their architectures. This is done in four parts.

First, we will see the study the influence of Inductive Biases and Feature Representations, our goal here is to see the influence of model architecture. By comparing the metrics of four well known and standard models (ResNet-50, ViT-B, CLIP). We will first perform a baseline metric on all of them, and then perform 5 different experiments. Colour Bias: We perform two different types, grey scale, and hue shift, and see how well the models perform. Shape vs Texture (cue-conflict): We apply the texture of one object to another, to see if models are using texture or shape for classification and which is more important. Translation: We shift the images, left, right, up, or down and see if the model can still classify. Lastly, we shuffle the patches of the images to see how the global structure effects the models.

Our points of interest are, what cue-conflict and colour reveal about reliance on shape, texture, and colour, and what coverage can tell us about shape bias. How the models respond when spatial structure is altered and what that tells us about the importance of locality, global organisation, and positional sensitivity. To what extent prediction-level changes agree with changes in feature representations, and which observed difference can reasonably be attributed to architecture as opposed to pretraining data, supervision, augmentation or model capacity.

2 Experimental Setup/Methods

2.1 Inductive Biases and Feature Representations

Models:

- torchvision ResNet-50 with ResNet50_Weights.IMAGENET1K_V2;

- torchvision ViT-B/16 with ViT_B_16_Weights.IMAGENET1K_V1;
- OpenCLIP ViT-B-32 with pretrained='openai'.

Dataset: STL-10

Configuration:

- Seed: 6304
- Optimizer: AdamW
- Learning Rate: 10^{-3}
- Weight Decay: 10^{-4}

For the Shape vs Texture we used AdaIN implementation, which follows Huang & Belongie (2017). the encoder/decoder architecture and the pretrained weights come from the public reimplementation naoto0804/pytorch-AdaIN, weights mirrored at BhupatiNadar/adain-style-transfer-models on HuggingFace.

2.2 Unsupervised Domain Adaptation

Data: PACS. SOurce - Photo, Art Painting, Cartoon. Target: Sketch Optimisation: AdamW, lr 10^{-4} , wd 10^{-4}

Source Only: Cross-entropy on the pooled source batch. (Also ERM for Domain Generalisation)
 DAN: $\mathcal{L} = \mathcal{L}_{cls} + \lambda \text{MMD}^2$ on 512-d features and kernel = sum of 3 RBFs with bandwidths $\{0.5, 1, 2\} \times$ median pairwise squared distance of the combined batch (bandwidth detached), $\lambda = 1$

2.3 Domain Generalisation

ERM is shared with Task 2. No sketch access.

DAN-DG: $\mathcal{L} = \mathcal{L}_{cls} + \frac{\lambda_{DG}}{3} \sum_{e < e'} \text{MMD}^2(F(X_e), F(X_{e'}))$

Sharpness Proxy: $\Delta_{\text{sharp}} = \mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta)$, $\epsilon = 0.05 g / \|g\|_2$

2.4 Open-Set Recognition

Data: CIFAR-10 train 50,000 $\rightarrow$ stratified 90/10: 45,000 train / 5,000 val, testing was done on both CIFAR 10 and CIFAR 100. Model: CIFAR ResNet-18: 3 x 3 stride-1 stem, no max-pool, random initialisation, 32 x 32 input, 512-d feature, Linear(512,10).

Scores: MSP = $1 - \max_k p_k$. MLS = $-\max_k z_k$. Energy = $-\log \sum_k e^{z_k}$. Mahalanobis = $\min_c (f - \mu_c)^\top \Sigma^{-1} (f - \mu_c)$

3 Results

3.1 Inductive Biases and Feature Representations

We first take some basic measurements as a baseline, we freeze the three classifier heads and measure top-1 accuracy, macro-F1, and mean maximum confidence. We also do this for zero shot CLIP, we measure confidence from the softmax over scaled class similarities.

Table 1: Clean Baseline - Shows all models performing very well on the dataset.

System	Top-1 (%)	Macro-F1 (%)	Mean max conf.
ResNet-50 + linear head	96.4	96.39	0.91
ViT-B/16 + linear head	97.2	97.21	0.97
CLIP ViT-B-32 + linear head	96.6	96.60	0.27
CLIP ViT-B-32 zero-shot	97.0	97.01	0.94

All the models perform well as a baseline, this maybe a limitation of the STL-10 dataset, as its distinct classes would be easier to classify. This can be a limitation as differences in experiments would more visibly show a decrease rather than an increase. It should also be noted that the CLIP Max Mean is low because it uses Cosine similarity in training, and the L-2 norm forces the magnitude to be 1. So when attached with a linear head and no compensation, the value comes off as low. It does not mean it is lower in confidence than other models, just that rows can't be compared the magnitude lower, not because it is less confident than others

3.1.1 Color Bias

We did two experiments with colour, the first we changed the images to greyscale measured the same metrics and got the following results:

Table 2: Greyscale: Minimal change in Top-1 Accuracy

System	Top-1	Macro-F1	Δ Top-1	Consistency	Conf.
ResNet-50 + linear head	94.6	94.61	-1.8	95.8	0.91
ViT-B/16 + linear head	95.4	95.41	-1.8	98.0	0.96
CLIP ViT-B-32 + linear head	93.8	93.78	-2.8	93.0	0.25
CLIP ViT-B-32 zero-shot	93.0	92.97	-4.0	94.0	0.93

We can see that making the image greyscale results in a slight decrease in accuracy throughout. Given that we have removed information from the image this is to be expected. A more interesting observation is that there was no significant difference between Convolution and Vision Transformers, though consistency is slightly higher for the Vision Transformer. CLIP had the most sever impact. Both of the readings are lower than either ResNet or ViT-B, and further zero-shot is the lowest. One plausible explanation for this is that the text embeddings had an influence on how accurately important colour is during pre training. For example if a prompt were to be "A picture of a Brown Dog," that would influenced both the visual encoder and the text encoder. It would also explain why zero-shot performed even worse. This though is not conclusive, and would be hard to confirm.

The second experiment was with a Hue Shift of 40 deg. A limitation of this is that a hue shift will only have a good effect on well saturated images. On less saturated images the change is very minimal, making the result less significant.

Table 3: Hue Rotation of 40: Minimal change in accuracy

System	Top-1	Macro-F1	Δ Top-1	Consistency	Conf.
ResNet-50 + linear head	94.4	94.42	-2.0	95.2	0.90
ViT-B/16 + linear head	95.8	95.79	-1.4	98.2	0.96
CLIP ViT-B-32 + linear head	94.8	94.78	-1.8	95.8	0.26
CLIP ViT-B-32 zero-shot	95.2	95.18	-1.8	95.6	0.93

Again we see a minimal change in accuracy, this time they are much closer to each other. No claims can be made as the difference is small enough that it can be considered noise. Again though, we will find that ViT is just slightly more consistent than other models. It is also of note that unlike greyscale, CLIP is much more in line with other models. One explanation for this is that in greyscale, we totally lost colour information, however with a rotation, the relationship between the colour of the objects is still the same, it has just been rotated. Hence, the loss is less with CLIP now.

3.1.2 Shape vs. Texture

For this comparison alpha strength of 1 was required, as lower values did not transfer texture well enough. We get the following results:

Table 4: Shape vs Texture: Convolutional models are not shape biased.

System	Balanced ($n = 220$)					All accepted ($n = 286$)	
	N_{sh}	N_{tex}	N_{oth}	Shape bias	Coverage	Shape bias	Coverage
ResNet-50 + linear head	54	52	114	50.94	48.18	57.78	47.20
ViT-B/16 + linear head	99	14	107	87.61	51.36	89.94	55.59
CLIP ViT-B-32 + linear head	116	12	92	90.62	58.18	91.72	59.09
CLIP ViT-B-32 zero-shot	117	7	96	94.35	56.36	95.81	58.39

99 Out of a total of 400 candidate images only 286 were of good enough quality, which were further
100 reduced to 220 to make each class balanced. As expected, the Vision Transformers have a considerable
101 shape bias, while the convolutional model does not. Though one point of note is that ResNet still
102 managed to score a 50.94 on the Shape Bias. This is considerably higher than the original Geirhos et
103 al. (2019) paper. The reason for this can be mostly likely presumed to be the biggest limitation of
104 this experiment.

105 That is that the images were filtered out manually. This introduced significant human bias in the
106 process making the data unreliable. The human eye is considered to be heavily biased towards shape,
107 I personally noticed as well that during the filtering it was much easier for me to tell when the shape
108 data was destroyed in comparison to when texture data was to sufficiently applied. It is reasonable to
109 then conclude the data had a bias towards higher quality shape images than texture images.

110 Another notable observation is that for ViT and ResNet both, there most quantitative category is
111 neither shape nor texture, and CLIP barely has more shape classified images. Making the coverage
112 very low, and limiting our ability to draw further conclusions from this data. This can again be
113 contributed to the low quality of the data. The AdaIN implementation resulted in images with quite
114 high distortion with our particular dataset, hampering the ability of the models to classify.

115 3.1.3 Patch Structure

116 We divided the images into 4x4 patches, and shuffled the patches. We feed the shuffled images back
117 in the model to measure the metrics:

Table 5: Patch Shuffling: Transformers suffer the most

System	Top-1 (%)	Macro-F1 (%)	Δ Top-1	Consistency (%)	Mean max conf.
ResNet-50 + linear head	89.0	89.06	-7.4	90.4	0.87
ViT-B/16 + linear head	90.4	90.52	-6.8	92.6	0.89
CLIP ViT-B-32 + linear head	84.4	84.18	-12.2	84.6	0.20
CLIP ViT-B-32 zero-shot	84.4	84.20	-12.6	84.2	0.82

118 When we shuffle the images we destroy the global structure of the image, i.e. the shape. For Vision
119 Transformers this is quite bad as they use the self attention mechanism and look at a global picture.
120 Understandably, they suffer the most in accuracy. Convolutional networks however only look at the
121 local area, making them much more resilient to this, further texture is preserved more, as each patch
122 can act like a texture patch and we saw in table4, they are much more biased towards texture.

123 The ViT-B and ResNet model are close in accuracy. This is a bit unexpected because we discussed
124 ResNet should perform noticeably better here. The likely culprit is that we used zero padding, max
125 pooling, stride 2, and we averaged the final 7x7 map which all contribute to either a lack of local
126 information or hampers its readability for the models.

127 One final thing of note is that CLIP performs noticeably worse than ViT, even though they share the
128 visual encoder (as a Vision Transformer), the fact that both zero-shot and linear head land at the same
129 number implies that its not the involvement of the text encoder in the prediction process. This suggest
130 that contrastive learning (with the text encoder) during pre training or some other stage had some
131 factor that made them more dependent factor.¹

¹We also saw more shape bias for them in table4, but it was not enough of a difference to warrant any conclusion

3.1.4 Representation Analysis

We have discussed most experiments now, but those mainly measured when the predictions changed and hence also changed the accuracy. But how the model sees and interprets the feature can change even if the prediction didn't. We measure the Cosine Similarity and t-SNE maps where relevant.

Table 6: Cosine similarities - Expected Results

Intervention	ResNet-50		ViT-B/16		CLIP ViT-B-32	
	I_T	Norm.	I_T	Norm.	I_T	Norm.
Grayscale	0.789	0.719	0.737	0.711	0.900	0.743
Translation (32 px)	0.934	0.912	0.943	0.938	0.970	0.923
Patch shuffle	0.718	0.626	0.649	0.615	0.728	0.299
Cue conflict vs. content	0.250	0.004	0.223	0.146	0.651	0.101
Cue conflict vs. style	0.216	-0.041	0.072	-0.020	0.580	-0.083
Random-pair floor	0.247	0.000	0.090	0.000	0.612	0.000

In terms of grayscale and translation we can see the Norm is rather high. This is consistent with our earlier hypothesis that models don't look at colour for classification. And since in translation most features are preserved (shape, texture, colour) and the image is only moved, they are very close to the original in representation.

In Patch shuffling the values are a bit lower (around 0.6), which is to be expected as the global structure is destroyed. What is more interesting is that CLIP and ViT show a large difference despite them both having a Vision Transformer. One plausible explanation is that since CLIP's tokens are twice as large as ViT, they are more dependent on global structure, and the representation drifts more. The larger token size could also mean that it covers the boundaries more between patches.

The t-SNE diagrams (Figures 4.2.3 in the appendix) agree with these results.

3.2 Unsupervised Domain Adaptation

3.2.1 Source Only ERM

We first train both the model and the classifier on the source domains using cross entropy, which results in these measurements.

Table 7: Source only ERM - the main domain gap identified

Domain	Accuracy (%)	Macro-F1 (%)
Photo (source, val)	97.30	96.85
Art Painting (source, val)	91.71	91.69
Cartoon (source, val)	95.74	96.00
Mean source	94.91	94.85
Worst source	91.71	91.69
Sketch (target)	61.59	63.09

As we can see in source only statistics, the accuracy on our source domains is very good, but when ran on an unknown domain (for us that is sketch) we notice a massive accuracy drop. Compared with the mean it is a -33 % drop, this is the main figure we try to lower in these experiments.

Table 8: Target Domain by class - high amounts of variation.

Class	Sketch accuracy (%)
Dog	21.11
Elephant	94.73
Giraffe	77.95
Guitar	70.07
Horse	46.32
House	83.75
Person	61.25

when we see the by class accuracy on the target domain, dog performs worst while elephant performs best. Dog is the largest class in the sketch data, we found dog’s 333 most common errors go to elephant, 167 to giraffe. It is probable it is not confusing animals and objects at least, it is confused within animal classes.

3.2.2 DAN - MMD Alignment

Table 9: DA with MMD - Slight improvement over Source Only

	Source-only	DAN ($\lambda_{\text{MMD}} = 1$)
Photo val acc (%)	97.30	95.80
Art Painting val acc (%)	91.71	93.41
Cartoon val acc (%)	95.74	95.95
Mean source acc (%)	94.91	95.05
Mean source macro-F1 (%)	94.85	94.93
Sketch acc (%)	61.59	64.44
Sketch macro-F1 (%)	63.09	60.55
Δ Sketch acc	—	+2.85
Domain separability (%)	100.00	81.87

If we observe the general by class results for MMD, we see a slight but still meaningful improvement over the sketch accuracy. Though the sketch F1 - macro decreases by a bit.

Table 10: MMD by Class - Good by class improvement

Class	Source-only (%)	DAN (%)	Δ
Dog	21.11	47.93	+26.82
Elephant	94.73	75.54	−19.19
Giraffe	77.95	61.62	−16.33
Guitar	70.07	74.18	+4.11
Horse	46.32	57.60	+11.28
House	83.75	100.00	+16.25
Person	61.25	86.25	+25.00

We see all classes improve in accuracy, except for giraffe and elephant, but given that in source only most of the errors in the dog class went to giraffe and elephant, it is likely they were over represented in the original model in the first place.

3.2.3 DANN - Adversarial Alignment

We now use adversarial learning to see how well it performs. And get the following results:

Table 12: Tracking Per Epoch - Loss was decreasing while Domain accuracy was going to 0.5

Epoch	Cls. loss	Domain loss	Domain acc.	Mean source F1 (%)
1	0.533	0.384	0.817	91.37
2	0.234	0.597	0.673	89.19
3	0.159	0.669	0.597	94.03
4	0.261	0.742	0.542	84.62
5	0.176	0.686	0.551	92.80
6	0.084	0.681	0.541	92.37
7	0.070	0.694	0.502	93.92
8	0.081	0.694	0.501	93.90

Selected epoch in bold. Chance domain accuracy is 0.5 and $\ln 2 = 0.693$.

Table 13: CDAN: Significant Improvement from before

	Source-only	CDAN
Photo val acc (%)	97.30	97.30
Art Painting val acc (%)	91.71	91.46
Cartoon val acc (%)	95.74	95.52
Mean source acc (%)	94.91	94.76
Mean source macro-F1 (%)	94.85	94.66
Sketch acc (%)	61.59	74.65
Sketch macro-F1 (%)	63.09	73.85
Δ Sketch acc	—	+13.06
Domain separability (%)	100.00	99.86

Table 11: DANN - The performances worsened

	Source-only	DANN
Photo val acc (%)	97.30	95.80
Art Painting val acc (%)	91.71	92.93
Cartoon val acc (%)	95.74	93.60
Mean source acc (%)	94.91	94.11
Mean source macro-F1 (%)	94.85	94.03
Sketch acc (%)	61.59	55.97
Sketch macro-F1 (%)	63.09	58.23
Δ Sketch acc	—	−5.62
Domain separability (%)	100.00	99.31

165 Interestingly not only is the performance on the sketch pictures worse than source, but the the
 166 predictions on source only is also worse. At first this might make it look like DANN didn't work, but
 167 after further investigation we find this:

168 The Loss was indeed decreasing, and the Domain Accuracy was going to 0.5., what this means is
 169 that even if we fool the discriminator. that does not necessarily mean an increase in performance.
 170 Perhaps the backbone managed to fool the discriminator before the discriminator got good enough
 171 to warrant an improvement. An increase in learning later or even just running more Epochs could
 172 improve metrics to match MMD or beyond.

173 3.2.4 CDAN - Class-Conditional Adversarial Alignment

174 To improve the scores from before we use the values of the prediction as well, we do a cross product
 175 with the latent and feed that to the discriminator.

The performance on the source only metric stays the same, but the accuracy on sketch images received a significant improvement. Meaning that even soft information about the final prediction was very helpful to the discriminator. Improving its performance which improved the backbone to more aggressively compete with it. However, it was not lower domain separability that did this, as domain separability essentially remained the same. It can be speculated that domain information is important as that helps the model to classify them.

3.2.5 Common Evaluation and Alignment Diagnostic

We compare the results overall:

Table 14: Diagnostic - Separability might not be the deciding factor

Method	Source val. acc. (%)			Mean source (%)		Sketch (%)		Δ Acc.	Sep. (%)
	Photo	Art	Cartoon	Acc.	F1	Acc.	F1		
Source-only	97.30	91.71	95.74	94.91	94.85	61.59	63.09	—	100.00
DAN	95.80	93.41	95.95	95.05	94.93	64.44	60.55	+2.85	81.87
DANN	95.80	92.93	93.60	94.11	94.03	55.97	58.23	−5.62	99.31
CDAN	97.30	91.46	95.52	94.76	94.66	74.65	73.85	+13.06	99.86

Sep. = domain separability: held-out accuracy of a logistic-regression probe ($C = 1$, balanced, 70/30 split, seed 6304) trained to tell source from target from the frozen features. Chance is 50%. Target labels were used only at this stage.

We would assume that bringing the domains closer and making them indistinguishable would be key, as the overlapping elements between the target domain and source domains would come into play. However aside from MMD which tries to directly to do this with the Loss function, a freshly trained logistical regressor can still separate the classes. And our best performance from CDAN barely shows any change, implying it might not be the most key factor.

Table 15: Per Class Comparison: Four classes share a trend

Class	Sketch accuracy (%)				Δ vs Source-only		
	Source-only	DAN	DANN	CDAN	DAN	DANN	CDAN
Dog	21.11	47.93	42.10	64.25	+26.82	+20.99	+43.14
Elephant	94.73	75.54	35.00	80.54	−19.19	−59.73	−14.19
Giraffe	77.95	61.62	51.53	62.95	−16.33	−26.42	−15.00
Guitar	70.07	74.18	98.36	89.31	+4.11	+28.29	+19.24
Horse	46.32	57.60	48.04	71.69	+11.28	+1.72	+25.37
House	83.75	100.00	100.00	100.00	+16.25	+16.25	+16.25
Person	61.25	86.25	98.12	99.38	+25.00	+36.87	+38.13

Table 16: Two most wrong - Similar pattern

True class	Source-only	DANN	CDAN
Dog	elephant 333, giraffe 167	person 397, house 18	person 139, elephant 62
Horse	giraffe 293, elephant 111	dog 204, person 166	dog 134, person 33
Elephant	person 21, giraffe 13	person 400, dog 49	person 65, dog 58
Giraffe	elephant 99, horse 34	person 164, dog 142	dog 175, horse 56

Two most frequent wrong predictions per class, with counts.

Further data shows us that four classes show interesting behavior. Dog (+20.99 to +43.14) and Person (+25.00 to +38.13) improve under every method, while Elephant and Giraffe degrade under every method. It almost seems like the model is trading the improvement in two for the others, no one improved all four.

193 3.2.6 Controlled Design Study

Table 17: Control study - too high a λ collapses the model

λ_{MMD}	Mean source F1 (%)	Sketch acc. (%)	Domain separability (%)
0 (source-only)	94.85	61.59	100.00
0.1	94.73	62.31	92.17
1	94.93	64.44	81.87
10	5.07	4.07	60.03

194 A higher λ means a higher penalty for difference between the source domain feature distribution, so
 195 increasing it should mean the model more aggressively tries to match the feature distributions. And
 196 we see exactly that for the first three rows, however $\lambda = 10$ completely collapses the model. It is
 197 such a sever penalty that all the distributions get so mixed up that the model can no longer tell any of
 198 them apart. F1 Mean on source alone is enough to disqualify 10 as optimal even without consulting
 199 target domain labels.

200 3.3 Domain Generalization

201 We ruse the same ERM Baseline as last time, refereed to in table7 to be efficient with our compute
 202 resource.

203 3.3.1 DAN-DG - Pairwise Source-Domain Alignment

204 In our initial configuration, which matched task with DAN for the sake of comparison, the model
 205 collapses.

Table 18: Comparison with ERM - complete model collapsed.

	ERM	DAN-DG ($\lambda_{\text{DG}} = 1$)
Photo val macro-F1 (%)	96.85	5.86
Art Painting val macro-F1 (%)	91.69	5.14
Cartoon val macro-F1 (%)	96.00	4.21
Mean source acc (%)	94.91	21.68
Mean source macro-F1 (%)	94.85	5.07
Worst source macro-F1 (%)	91.69	4.21
Sketch acc (%)	61.59	4.07
Sketch macro-F1 (%)	63.09	1.12
Δ Sketch acc	—	−57.52
Source-domain separability (%)	91.21	38.74
Δ_{sharp}	0.2095	0.0433

Table 19: Step Data - Model is destroyed by step 50.

Step	λ_{DG}	Cls. loss	MMD	$\ f\ $	Feature std
0	1	2.042	0.725	27.93	0.7891
50	1	1.948	0.286	3.43	0.0017
100	1	1.932	0.625	2.37	0.0003
150	1	1.925	0.525	2.10	0.0000
199	1	1.959	0.748	2.03	0.0000
199	0	0.096	—	61.48	1.6616

Identical code and seed; only λ_{DG} differs. $\ln 7 = 1.946$ is the cross-entropy of a uniform predictor over seven classes.
 Gradient clipping at 1.0 did not prevent it.

206 We speculate that this happens due to the model too aggressively trying to align the source domains.
 207 So to remedy this we try to ramp up the penalty term (λ) from zero to one as the model trains
 208 instead of just starting from one. The slow increase should allow us the model to stabilise while still
 209 encouraging maximum alignment due to the increasing λ . Which brings us the following data.

Table 20: λ Ramp - Improvement from before but still collapsing

Variant	λ_{DG}	Source (%)		Sketch (%)		Sep. (%)	Δ_{sharp}
		Mean F1	Worst F1	Acc.	Δ		
ERM (no alignment)	0	94.85	91.69	61.59	—	91.21	0.2095
As specified	1	5.07	4.21	4.07	−57.52	38.74	0.0433
Reduced weight	0.1	96.78	96.29	69.81	+8.22	72.80	0.1394
Increased weight	10	5.07	4.21	4.07	−57.52	27.47	0.0057
λ ramp	0 to 1	94.43	92.41	62.31	+0.72	76.37	0.3216
Bandwidth gradient	1	94.45	92.56	71.49	+9.90	70.05	0.2400

210 It is at least enough to stabilise the the model, however we see little to no improvement in the target
 211 domain. So as a last resort we increase the Bandwidth Gradient to 1 in the RBF Kernel, this is perhaps
 212 the most direct way of telling the MMD Loss to not bring the domains too close, this finally results in
 213 a significant improvement to the sketch accuracy, however we can no longer compare these results to
 214 our earlier experiments. There was more room for improvement here, we could perhaps try a fixed λ
 215 value of 0.5, however due to limited compute times, we could not try that.

216 3.3.2 SAM - Parameter-Space Stability

217 We now try to land on a flatter space of the loss, which should in theory be where the neighboring
 218 space all occupy similar features.

Table 21: SAM - Stable performances and great improvement

	ERM	SAM ($\rho = 0.05$)
Photo val macro-F1 (%)	96.85	96.41
Art Painting val macro-F1 (%)	91.69	93.65
Cartoon val macro-F1 (%)	96.00	97.13
Mean source acc (%)	94.91	95.74
Mean source macro-F1 (%)	94.85	95.73
Worst source macro-F1 (%)	91.69	93.65
Sketch acc (%)	61.59	74.12
Sketch macro-F1 (%)	63.09	74.98
Δ Sketch acc	—	+12.53
Source-domain separability (%)	91.21	89.01
Δ_{sharp}	0.2095	0.1116

219 This worked much better and cleaner than our DAN-DG we see a obvious improvement on our target
 220 domain.

221 3.3.3 Common Evaluation and Diagnosis

222 We compare our previous results.

Table 22: Full Comparison - source

Method	Source val. acc. (%)			Mean source (%)		Worst (%)		Sketch (%)		Δ Acc.
	Photo	Art	Cartoon	Acc.	F1	Acc.	F1	Acc.	F1	
ERM	97.30	91.71	95.74	94.91	94.85	91.71	91.69	61.59	63.09	—
DAN-DG	96.40	92.93	94.88	94.74	94.45	92.93	92.56	71.49	73.20	+9.90
SAM	97.00	93.41	96.80	95.74	95.73	93.41	93.65	74.12	74.98	+12.53

ERM is Task 2’s source-only checkpoint, reused unchanged. SAM uses the specified $\rho = 0.05$. DAN-DG uses $\lambda_{DG} = 1$ with gradient flowing through the median-heuristic bandwidth; with the bandwidth detached the run collapsed (Table 19).

We compare all our numbers against each other, with one interesting trend that was also in Domain Adaptation, which that accuracies on source domains are not at all predictive of sketch accuracy, difference between source being minimal with SAM and ERM only, but sketch accuracy is massively improved by SAM.

Table 23: Domain Separability and Sharp - SAM had lowest Domain separability

Method	Source-domain separability (%)	Δ_{sharp}
ERM	91.21	0.2095
DAN-DG	70.05	0.2400
SAM	89.01	0.1116

Separability: three-way logistic-regression probe over Photo / Art Painting / Cartoon on frozen features, $C = 1$, 70/30 split, seed 6304; chance is 33.3%. Δ_{sharp} : cross-entropy increase after one normalised ascent step of radius 0.05 on a fixed batch of 32 images per source domain, model in eval mode.

We see that DAN does the most work for Source Domain Separability which is what MMD targets, while SAM does not really touch for domain separability but does reduce sharpness the most as we want it to. SAM performs very well compared to DAN.

3.3.4 Controlled Desgin Study

We varied the ρ values for SAM to find how far the trend extends

Table 24: ρ Values – Collapses on 0.1

ρ	Source F1 (%)		Sketch (%)		Sep. (%)	Δ_{sharp}
	Mean	Worst	Acc.	Δ		
0 (ERM)	94.85	91.69	61.59	—	91.21	0.2095
0.01	94.59	91.48	63.58	+1.99	90.38	0.1702
0.05	95.73	93.65	74.12	+12.53	89.01	0.1116
0.1	12.75	10.68	4.07	−57.52	67.86	0.0091

with high ρ values the bound extends so far that it starts to go to other regions as well. Sharp still increases as the shallowed are becomes a subset, but performance is completely destroyed which slowest that it can’t be used as an accurate independent measure of performance for SAM.

3.4 Open-Set Recognition (OSR)

3.4.1 Vanilla Closed-Set Baseline

We first train a very traditional ResNet classifier, with ten classes to get a Vanilla score. The results are as follows.

Table 25: Vanilla Model - Intentionally made strong with 95 validation accuracy for later comparison.

Vanilla ResNet-18	
Epochs	100
Selected epoch	93
CIFAR-10 train accuracy (%)	100.00
CIFAR-10 validation accuracy (%)	95.08
CIFAR-10 test accuracy, CSA (%)	94.26

CIFAR-10 known classes; 45,000 / 5,000 stratified train / validation split, seed 6304. Checkpoint selected by validation accuracy only.

3.4.2 Post-Hoc Novelty Score

We froze the Vanilla Model, and measured different unknownness scores on it.

Table 26: TODO: your caption and key take-away.

Score	τ	Test acc. (%)	Near rej. (%)	Far rej. (%)	FPR@95TPR near / far (%)
MSP	0.1721	94.34	28.25	45.38	71.75 / 54.62
MLS	-5.7175	94.77	30.62	54.62	69.38 / 45.38
Energy	-5.9177	94.84	31.38	54.88	68.62 / 45.12
Mahalanobis	2764.0347	94.62	27.25	50.12	72.75 / 49.88

τ is the 95th percentile of unknownness on the CIFAR-10 validation set; accept when $u(x) \leq \tau$. Rejection is the share of unknowns not accepted; FPR@95TPR is the share incorrectly accepted.

3.4.3 GCSC - Strong Closed-Set Classifier + MLS

Training our model extremely well on a closed set should make it so that it'll be able to differentiate unknowns regardless. We achieve this by training on random positive augmentations.

Table 27: GCSC - Validations acc and Test acc both increase slightly

	Vanilla	GCSC
Selected epoch	93	98
CIFAR-10 validation accuracy (%)	95.08	95.62
CIFAR-10 test accuracy, CSA (%)	94.26	95.38

Identical recipe, initialization, optimiser, schedule, batch size, epochs and seed; GCSC adds RandAugment(2, 9) after the crop and flip.

For CIFAR 10 we see that both validation and test accuracy increase slightly, but when we see the effect on unknown classifications, we see that they also improve across the board.

Table 28: GCSC - Classifications scores all improve

Model	AUROC near (%)	AUROC far (%)	AUROC all (%)	CSA (%)
Vanilla + MLS	79.42	89.84	84.63	94.26
GCSC + MLS	81.84	92.13	86.99	95.38
Δ	+2.42	+2.29	+2.36	+1.12

3.4.4 PROSER - Classifier and Data Placeholders

We create a new class by taking latents of two classes and doing an inner join to get their middle, this creates a new unknown class.

Table 29: PROSER - Same Level of CIFAR-10 for less EPOCH

	Vanilla	GCSC	PROSER
Epochs	100	100	50
Selected epoch	93	98	14
CIFAR-10 validation accuracy (%)	95.08	95.62	94.72
CIFAR-10 test accuracy, CSA (%)	94.26	95.38	93.63

PROSER is initialised from the selected Vanilla checkpoint with five dummy classifiers appended. CSA uses the ten known-class logits only.

On the CIFAR-10 data set we see similar levels of performance, though we get there earlier, the same trend continues for Unknown classifications.

Table 30: PROSER - Same Level of performance for less EPOCH.

Model	Score	AUROC near (%)	AUROC far (%)	AUROC all (%)	CSA (%)
Vanilla	MLS	79.42	89.84	84.63	94.26
GCSC	MLS	81.84	92.13	86.99	95.38
PROSER	MLS	77.61	87.57	82.59	93.63
PROSER	Placeholder	74.36	84.82	79.59	93.63

3.4.5 Common Evaluation and Failure Analysis

Table 31: Unknownness Score Types - Mahalanobis performed worse while Energy was Best

Score	AUROC (%)			At the 95% validation threshold (%)			
	Near	Far	All	Test accept.	Near rej.	Far rej.	FPR near / far
MSP	80.98	89.66	85.32	94.34	28.25	45.38	71.75 / 54.62
MLS	79.42	89.84	84.63	94.77	30.62	54.62	69.38 / 45.38
Energy	79.43	89.92	84.67	94.84	31.38	54.88	68.62 / 45.12
Mahalanobis	79.00	91.22	85.11	94.62	27.25	50.12	72.75 / 49.88

Test accept. is the share of CIFAR-10 test images accepted at the threshold.

All Unknownness score types had similar performance, in the Near rej stats, which would be the hardest, Mahalanobis performed the worst while Energy performed the best.

Table 32: Different Models - GCSC performs the best, both PROSER are worse than Vanilla

Model	Score	CSA	AUROC (%)			At the 95% threshold (%)		
			Near	Far	All	Test accept.	Near rej.	Far rej.
Vanilla	MLS	94.26	79.42	89.84	84.63	94.77	30.62	54.62
GCSC	MLS	95.38	81.84	92.13	86.99	94.35	37.62	64.00
PROSER	MLS	93.63	77.61	87.57	82.59	94.58	29.38	44.50
PROSER	Placeholder	93.63	74.36	84.82	79.59	94.66	27.25	41.37

CSA is closed-set accuracy on the ten known-class logits, so both PROSER rows share it. Test accept. is the share of CIFAR-10 test images accepted at the threshold.

GCSC is the best performer, with significant improvement over Vanilla, both PROSER variants perform worse than Vanilla, the PROSER MLS can be chalked by to Non Gain from the Vanilla. The -1.25 gap could be noise. but PROSER Placeholder shows amore apprent worse score. This is more apprent in Far rejections, where both perform considerably worse than Vanilla. We choose EPOCH 14 as that is the CIFAR 10 validation was highest, but at that early stage it is possible that the PROSER simply didn'T traing enough to make significant progress, and the extra unknown class hampered the model's ability (as the logits would now be spread on one additional class), instead of helping it.

Table 33: Failure Rates - Far are much easier than near

Group	Unknown class $\rightarrow$ predicted class	MLS score	τ	Accepted in class
Near	pickup_truck $\rightarrow$ automobile	-11.54	-5.72	91/100
Near	bus $\rightarrow$ truck	-12.36	-5.72	82/100
Near	wolf $\rightarrow$ dog	-11.37	-5.72	81/100
Far	wardrobe $\rightarrow$ cat	-11.53	-5.72	58/100
Far	mushroom $\rightarrow$ cat	-12.73	-5.72	55/100
Far	sunflower $\rightarrow$ bird	-10.99	-5.72	52/100

Vanilla, MLS; accept when the score is $\leq \tau$. Each row is the most confidently accepted image of that unknown class; the last column gives how many of its 100 test images were accepted.

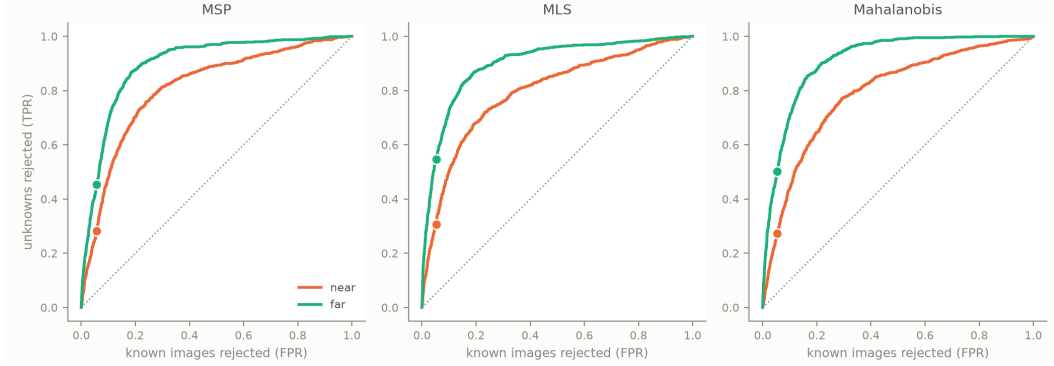


Figure 1: ROC curve for Detector - Far always performs better

262 Detecting Far images seems to be much easier throughout, most likely because they look so different,
 263 so it's easier for the model to tell them apart. In table33 we can see that where as the worst Near are
 264 all wrongly classified at above 80%, the far are all less than 60%.

265 3.5 Cross-Task Discussion

266 4 Conclusions

267 5 References

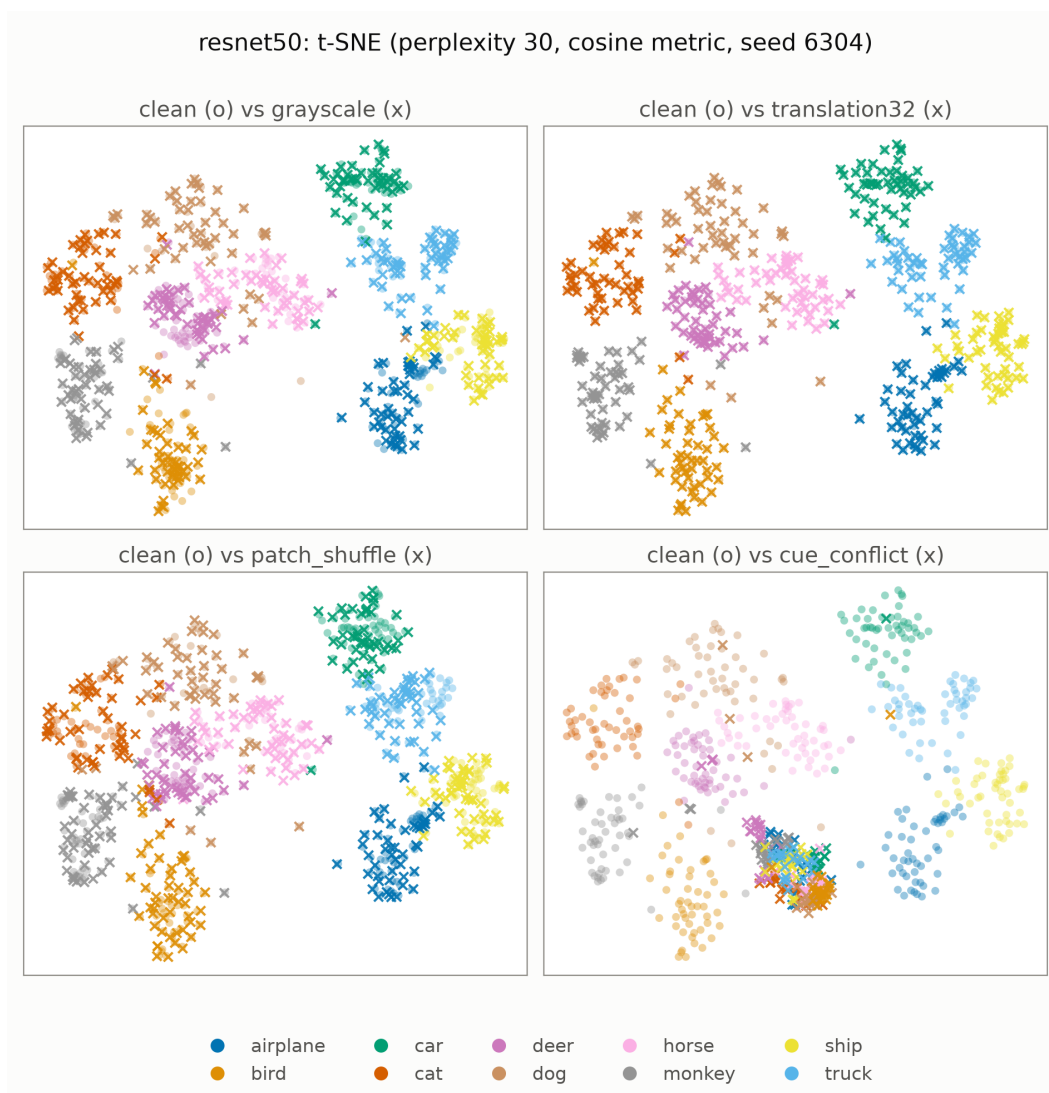


Figure 2: ResNet t-SNE: Representations overlap except in Cue Conflict

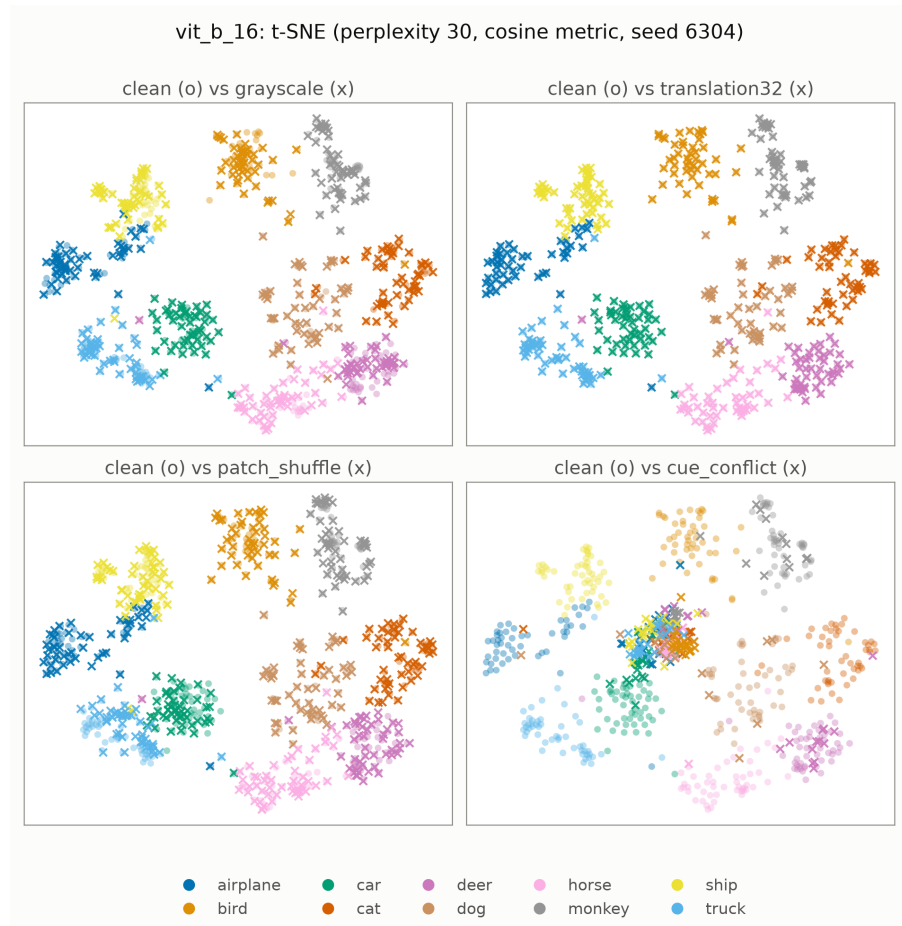


Figure 3: ViT-B t-SNE: Representations overlap except in Cue Conflict

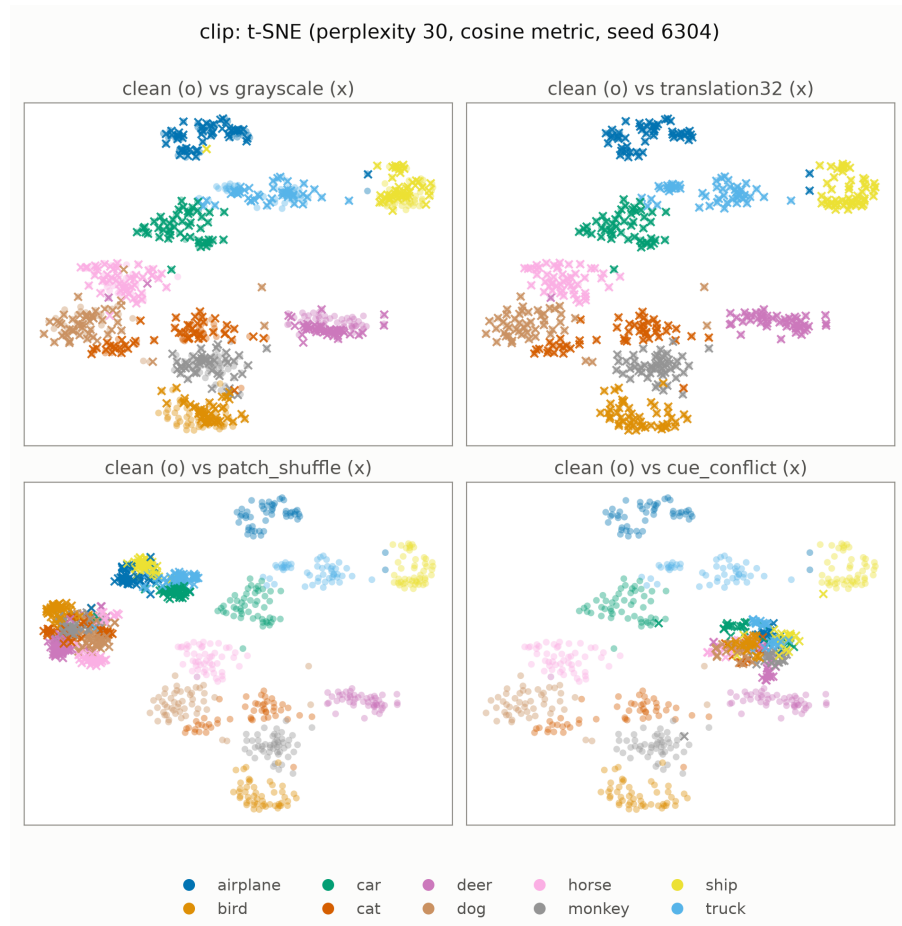


Figure 4: CLIP t-SNE: Representations overlap except in Cue Conflict and Patch Shuffle